

Absolutely. Let's move to **Set 4 — harder production and architecture scenarios**. These are especially useful for a **4+ years GenAI/RAG interview**, where the interviewer may ask **"What would you do if..."**

■ GenAI 4+ Years Interview — Set 4

151. Your RAG answers are correct for short questions but wrong for long questions. What do you check?

Answer:

"I would check whether the long query is being embedded effectively and whether important parts of the query are being lost. I would use query rewriting or query decomposition and then retrieve information for each sub-question."

152. User asks a question containing multiple questions. What do you do?

Example:

"What is S3, how does versioning work, and what are its storage classes?"

Answer:

"I would decompose the query into smaller sub-questions, retrieve relevant context for each one, and then combine the results into a single response."

Complex Question

↓

Question Decomposition

↓

■■■■■■■■■■■■■■■■■■■■

Q1 Q2 Q3

↓↓↓

Retrieval for each

↓

LLM

↓

Combined Answer

153. Why is query decomposition useful?

Answer:

"Complex questions may require information from different parts of the knowledge base. Decomposition improves retrieval precision by searching separately for each part."

154. Your documents are updated every hour. How do you keep RAG current?

Answer:

"I would implement an automated ingestion pipeline that detects new or changed documents, processes only those documents, generates new embeddings, and updates the vector database. I would also maintain document versions and timestamps."

155. How do you detect changed documents?

Answer:

"I can use document IDs, modification timestamps, version numbers, or content hashes. A content hash is useful for detecting whether the actual document content has changed."

156. What if 10,000 documents change at the same time?

Answer:

"I would process them asynchronously using a queue and worker architecture. I would batch embedding operations, scale workers horizontally, track progress, retry failures, and avoid blocking the user query service."

157. Why use asynchronous processing during ingestion?

Answer:

"Embedding thousands of documents is a long-running operation. Asynchronous processing prevents it from blocking API requests and allows the workload to be distributed across multiple workers."

■ Queue & Worker Scenarios

158. Why use a message queue in GenAI?

Answer:

"A queue decouples producers from consumers and helps handle spikes in workload. For example, document uploads can enter a queue and worker processes can perform extraction, chunking, embedding, and indexing asynchronously."

159. What happens if an ingestion worker crashes?

Answer:

"The job should remain retryable. I would use acknowledgments, retry mechanisms, and a dead-letter queue for persistent failures."

160. What is a dead-letter queue?

Answer:

"A dead-letter queue stores messages that repeatedly fail processing so they can be investigated without blocking the rest of the workload."

161. Your queue has 1 million documents waiting. What do you do?

Answer:

"I would monitor queue depth, increase worker capacity, use batch processing, auto-scale workers, and prioritize workloads if necessary."

■ Multi-Document RAG

162. A question requires information from five different documents. Can RAG handle it?

Answer:

"Yes. The retriever can return chunks from multiple documents. I would preserve document metadata and ask the LLM to synthesize the answer from the retrieved sources."

163. One document says "5 days" and another says "10 days." What should the chatbot do?

Answer:

"I would use metadata such as version, effective date, and authority level to determine which document is current. If the conflict cannot be resolved, I would tell the user that the sources conflict rather than inventing an answer."

164. How would you cite sources in a RAG answer?

Answer:

"I would preserve document ID, file name, page number, and other metadata with each chunk. After generation, I would map the answer back to the retrieved source metadata and display citations."

165. Why are citations useful?

Answer:

"They improve trust and allow users to verify the answer against the original document. They are especially important for enterprise and regulated use cases."

■ Document Processing

166. A PDF has 600 pages and takes too long to process. What do you do?

Answer:

"I would process it asynchronously and in batches rather than loading the entire PDF into memory. I would monitor processing time and memory usage and parallelize independent pages or documents where appropriate."

167. A PDF contains scanned pages. What do you do?

Answer:

"I would use OCR to extract the text before chunking and embedding. I would also validate OCR quality because incorrect extraction will directly affect retrieval quality."

168. A PDF contains charts and graphs. How would you handle them?

Answer:

"If the information in the charts is important, I would use a document-processing or multimodal pipeline capable of extracting information from images or charts. I would store the extracted information with appropriate metadata."

169. Tables are getting corrupted during PDF extraction. What do you do?

Answer:

"I would use a table-aware extraction tool rather than treating the table as normal text. I would validate the extracted rows and columns before indexing."

■ Vector Database

170. Why did you choose FAISS?

Answer:

"FAISS provides efficient similarity search over vector embeddings and is useful for local or controlled deployments. It is simple and fast for prototyping and smaller-scale workloads."

171. When would you move from FAISS to a managed vector database?

Answer:

"When the application requires distributed scaling, persistence, high availability, metadata filtering, multi-user access, or operational features that are easier to manage with a dedicated vector database."

172. What happens if your vector database becomes unavailable?

Answer:

"I would use monitoring and health checks to detect the failure. Depending on the architecture, I would fail over to a replica or rebuild the index from the source documents. The application should return a graceful error rather than produce unsupported answers."

173. How do you back up a vector database?

Answer:

"I would maintain backups of the vector index and, more importantly, preserve the original documents and metadata so the embeddings can be regenerated if necessary."

174. Why should you keep the original documents if you have embeddings?

Answer:

"Embeddings are derived representations, not the source of truth. If we change the embedding model, chunking strategy, or vector database, we need the original documents to rebuild the index."

■ Embedding Scenarios

175. What happens if query and document embeddings use different models?

Answer:

"Retrieval can become invalid because the vectors may not be compatible. Query and document embeddings should normally be generated using the same compatible embedding space."

176. Why can't you simply store raw text in a vector database?

Answer:

"Vector search works on numerical representations of semantic meaning. The text needs to be converted into embeddings before performing vector similarity search."

177. What does an embedding represent?

Answer:

"An embedding is a numerical vector representation of text that captures semantic characteristics. Texts with similar meanings tend to have vectors that are closer in the embedding space."

■ Similarity Search

178. What is cosine similarity?

Answer:

"Cosine similarity measures the angle between two vectors. A higher cosine similarity generally means the vectors are more semantically similar."

...

179. Why not just use keyword search?

Answer:

"Keyword search works well for exact terms but can miss semantically similar phrases. Vector search captures meaning, while hybrid search can combine both approaches."

...

180. When is keyword search better than vector search?

Answer:

"For exact identifiers such as error codes, product IDs, ticket numbers, names, or specific technical terms, keyword search can be more effective."

...

■ Hybrid Search

181. What is hybrid search?

Answer:

"Hybrid search combines keyword-based retrieval with semantic vector retrieval and combines their results to improve search quality."

User Query

↓

[illegible]

↓ ↓

Keyword Search Vector Search

↓ ↓

XXXXXXXXXXXXXXXXXXXX

↓

Ranking

↓

LLM

182. When would you choose hybrid search?

Answer:

"When the application needs both semantic understanding and exact matching. Technical documentation, error codes, product IDs, and enterprise documents are good examples."

■ Guardrails

183. What are guardrails in GenAI?

Answer:

"Guardrails are controls that restrict or validate model behavior. They can enforce topics, safety rules, access control, input validation, output validation, and response formats."

184. How would you prevent the chatbot from answering outside its domain?

Answer:

"I would define the allowed domain in the system prompt, use intent classification or a router, and reject or redirect out-of-scope questions. I would also ensure the RAG retrieval only uses approved sources."

185. User asks:

"Write me a poem."

Your application is an HR policy chatbot. What should happen?

Answer:

"The system should recognize that the request is outside the application's intended scope and return a controlled response instead of unnecessarily querying the HR knowledge base."

■ Data Privacy

186. Should you send confidential company documents to a public LLM API?

Answer:

"Only if the organization's security and data-processing policies explicitly allow it. Otherwise, I would use an approved enterprise model, private deployment, or appropriate data protection controls."

187. How do you protect sensitive data sent to an LLM?

Answer:

"I would minimize the data sent, apply access control, mask sensitive information where appropriate, use encryption, follow the organization's data retention policies, and use an approved model/provider."

188. How do you prevent sensitive information from appearing in logs?

Answer:

"I would avoid logging full prompts and responses by default, mask PII and secrets, apply structured logging, and restrict access to logs."

■ Agentic AI Scenarios

189. What is the difference between RAG and an AI agent?

Answer:

"RAG primarily retrieves information and provides it to an LLM for answering. An agent can reason about a task, choose tools, execute actions, observe results, and continue until the task is completed."

190. When would you use an agent instead of simple RAG?

Answer:

"If the application needs multi-step reasoning or actions such as querying a database, calling APIs, checking systems, and then taking an action, an agent may be appropriate. If the requirement is simply answering questions from documents, RAG is usually simpler."

191. Your agent keeps calling the same tool repeatedly. What do you do?

Answer:

"I would add iteration limits, tool-call validation, timeouts, state tracking, and clear stopping conditions."

192. An AI agent calls the wrong API. How do you prevent it?

Answer:

"I would use strict tool schemas, authorization, input validation, tool descriptions, confirmation for high-risk actions, and logging. The agent should only have access to the minimum tools and permissions required."

■ Final Advanced Production Questions

193. How would you handle a P1 issue in a GenAI application?

Answer:

"I would first assess impact and stabilize the service. I would check monitoring, logs, recent deployments, LLM/API health, vector database health, and infrastructure. If necessary, I would roll back the latest change or enable a fallback. After recovery, I would perform root-cause analysis and implement preventive measures."

194. Production RAG suddenly starts returning "I don't know" for everything. What do you check?

Answer:

"I would check whether the vector database is available, whether embeddings are being generated correctly, whether the similarity threshold changed, whether the index contains documents, and whether metadata filters are incorrectly excluding documents."

195. RAG suddenly retrieves zero documents after a deployment. What is your first check?

Answer:

"I would check the retrieval logs, vector database connectivity, embedding generation, index availability, similarity threshold, and metadata filters. I would compare the current configuration with the previous working version."

196. Your LLM response quality suddenly drops but retrieval is unchanged. What do you check?

Answer:

"I would check the model version, prompt version, generation parameters, context formatting, and output-token configuration. I would compare the current deployment with the last known good version."

197. How would you roll back a bad GenAI deployment?

Answer:

"I would use versioned code, prompts, models, and configuration so that I can quickly return to the last known good version. I would monitor the rollback and validate the system using the evaluation dataset."

198. How do you perform A/B testing for prompts?

Answer:

"I would route a controlled percentage of traffic to each prompt version and compare metrics such as correctness, groundedness, user feedback, latency, and cost."

199. How do you decide whether a RAG improvement is actually better?

Answer:

"I would compare it against a fixed evaluation dataset and production metrics. I would check retrieval quality, answer correctness, groundedness, latency, and cost rather than judging it from a few examples."

200. Interviewer: "Explain your production RAG architecture."

■ Strong 4+ years answer

"I would have two major pipelines: document ingestion and user query processing.

 $\succ$

During ingestion, documents are uploaded, validated, extracted, cleaned, and split into chunks. I generate embeddings for each chunk and store them in a vector database along with metadata such as document ID, version, department, access level, and page number.

>

During query processing, the user is authenticated and authorized first. The question is converted into an embedding and used for similarity or hybrid search with metadata filtering. I retrieve candidate chunks, apply a similarity threshold and reranking, and send only the most relevant context to the LLM. The prompt instructs the LLM to answer only from the provided context and say 'I don't know' when sufficient information is unavailable.

 $\geq$

For production, I would add caching, rate limiting, retries with exponential backoff, timeouts, circuit breakers, load balancing, auto-scaling, monitoring, logging, evaluation, security controls, and cost tracking."

■ The architecture to remember

DOCUMENTS

■

▼

Document Ingestion

Extract / Clean

■

Chunk

■

Embeddings

■

▼

[illegible]

■ Vector Store ■

■ + Metadata ■

[REDACTED]

■

■

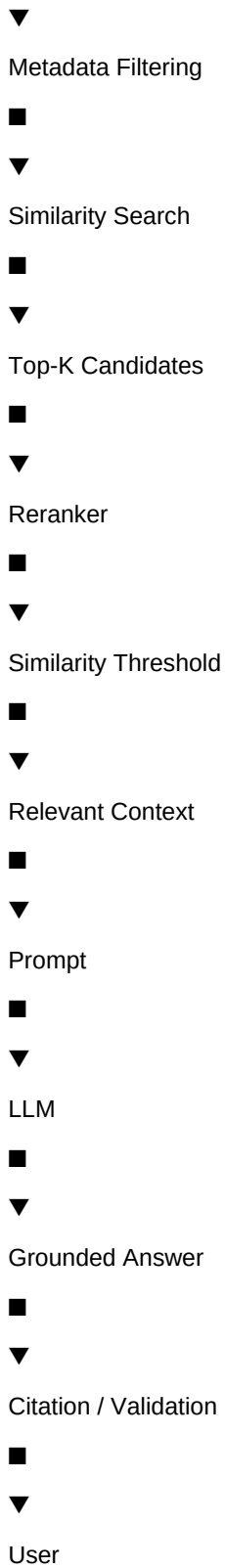
USER ■■■ Authentication ■■■ Authorization

■

▼

Query Embedding

■



The key mindset for a 4+ years interview is: don't just say "I will use RAG."* Explain how you would detect the problem, which component you would inspect, what change you would make, and how you would measure that the change worked.